SISTEMAS DIGITALES · Unidad 1: Microcontroladores para sistemas embebidos · Semana 2 · Sesión 2

Programar para el hardware

Tipos de datos, tiempo sin bloquear y pulsadores

1 1 0 0 1 1 0 0

Ing. Luis Sinche Cueva, Mg. · Carrera de Computación – UNL · Octubre 2026

Objetivo de la sesión

Al finalizar, el estudiante podrá:

1. Elegir tipos de datos de tamaño explícito y prever desbordamientos y promociones.
2. Implementar temporización no bloqueante con millis().
3. Leer un pulsador con pull-up, antirrebote y detección de flanco.

Resultado de aprendizaje

R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.

Todo lo de hoy se usa el viernes en la AP2: Juego de luces interactivo.

Activación: ¿qué imprime?

```
uint8_t a = 0xCC;          // 1100 1100
Serial.println(~a, BIN);   // ¿?
```

Pista: ¿cuántos bits tiene un int?

Escriban su predicción:

_ _ _ _ _ _ _ _ …

Es la pregunta 6 de la AP1. Hoy entenderemos por qué.

1

Parte 1 · Tipos de datos

Una variable es una caja en la SRAM

Una variable es una caja: tamaño e interpretación



1 byte en la SRAM

uint8_t

Sin signo
= 204

int8_t

Con signo (bit alto = signo)
= −52

El tipo define:

- **el tamaño de la caja**
- **cómo se leen sus bits**

int no mide lo mismo en todas partes

Tipo	AVR (Uno)	ESP32	Ojo
`char`	1 · con signo	1 · sin signo	usar int8_t / uint8_t
`short`	2	2	
`int`	2	4	¡cambia de tamaño!
`long`	4	4	
`float`	4	4 (FPU)	rápido en el ESP32
`double`	4 (= float)	8 (software)	lento en el ESP32
`puntero`	2	4	

Tamaños en bytes. ESP32 verificado con el core Espressif 3.0.7.

stdint.h: el tamaño va en el nombre

`int8_t`

−128 … 127

`uint8_t`

0 … 255

`int16_t`

−32 768 … 32 767

`uint16_t`

0 … 65 535

`int32_t`

≈ ±2,1 × 10^9

`uint32_t`

0 … ≈ 4,29 × 10^9

Usen tipos de stdint.h para:

- registros y máscaras
- protocolos y tramas
- arreglos grandes
- tiempos (uint32_t)

u = sin signo · número = bits · _t = tipo

Desbordamiento: el odómetro que da la vuelta



`uint8_t c = 253;  c++ cinco veces`

Sin signo

Da la vuelta.
Comportamiento definido.

Con signo

Comportamiento indefinido.
No confiar en él.

int t = millis();

Uno: falla a los ~32 s

ESP32: falla a los ~24,8 días

Usar uint32_t

Demo: detector de plataforma

```
#define MOSTRAR(T) do { Serial.print(#T); Serial.print(": "); \
    Serial.print(sizeof(T)); Serial.println(" bytes"); } while (0)

void setup() {
  Serial.begin(115200);
  MOSTRAR(char); MOSTRAR(int); MOSTRAR(long);
  MOSTRAR(float); MOSTRAR(double); MOSTRAR(void*);

  uint8_t c = 253;                              // desbordamiento
  for (int i = 0; i < 5; i++) { Serial.print(c++); Serial.print(' '); }
  Serial.println();

  uint8_t a = 0xCC;                             // promoción
  Serial.println(~a, BIN);
  Serial.println((uint8_t)~a, BIN);
}
void loop() {}
```

Mitad del curso

Arduino Uno en Wokwi

Otra mitad

ESP32 DevKit en Wokwi

Máscaras: el 1 también tiene tipo

✗ Riesgoso

```
uint32_t m = 1 << 31;   // int con signo
```

Desborda un int con signo

✓ Correcto

```
uint32_t m = 1UL << 31; // sin signo
```

Literal de 32 bits sin signo

Promoción a int

```
(uint8_t)~a   // vuelve a 8 bits
```

~a sin cast da 32 bits

Literales:
0x hex · 0b binario
U sin signo · UL unsigned long

Constantes con tipo

Forma	Qué es	¿Tipo?	Uso
`#define LED 16`	reemplazo de texto	no	solo por compatibilidad
`const uint8_t LED = 16;`	constante verificada	sí	✓
`constexpr uint8_t LED = 16;`	calculada al compilar	sí	✓ la usaremos
`enum Modo { LENTO, RAPIDO };`	estados con nombre	sí	✓ modos del juego

En el ESP32, una const global se queda en la flash. En el AVR se copia a la SRAM, salvo PROGMEM o F().

volatile y static

volatile

«Este valor puede cambiar sin que lo veas».
Para variables que modifica una interrupción o el hardware.
No significa atómico.

static

Dentro de una función: conserva su valor entre llamadas.
En un archivo: privada de ese archivo.

Las variables locales viven en la pila de loopTask (8 KB): un arreglo local grande la desborda y el ESP32 se reinicia.

Actividad: encuentra el bug

A

```
for (uint8_t i = 0; i < 256; i++)
  Serial.println(i);
```

B

```
int t0 = millis();
```

C

```
uint32_t m = 1 << 31;
```

D

```
void loop() {
  int veces = 0;
  veces++;
}
```

En parejas: ¿cuál es el error y cómo se corrige? Tienen 6 minutos.

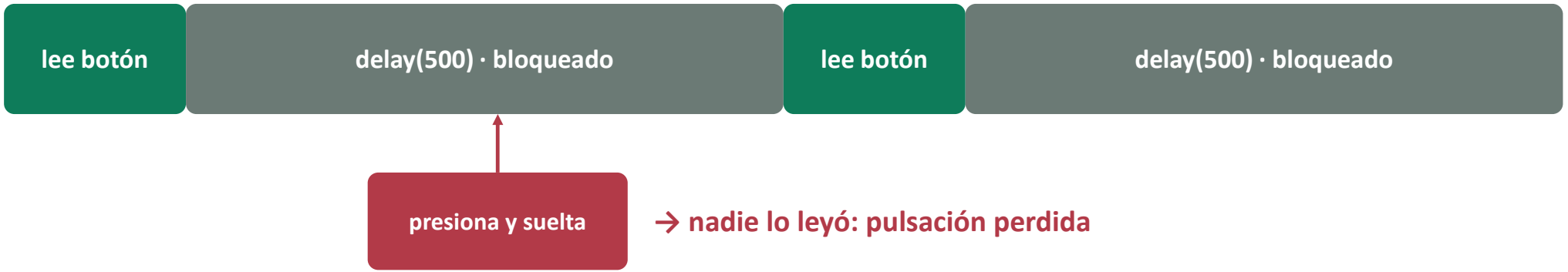
2

Parte 2 · El tiempo sin bloquear

delay() frente a millis()

delay(): el cocinero que mira el horno

loop() con delay(500)



delay() bloquea: mientras espera, el programa no hace nada más.
En un juego de luces, el botón «no responde».

millis(): mirar el reloj de pared

```
uint32_t ahora = millis();

if (ahora - tAnterior >= PERIODO) {
  tAnterior += PERIODO;
  // hacer algo
}
// el loop sigue: hay tiempo para leer botones
```

El patrón

1. Leer el reloj
2. Restar: ¿cuánto pasó?
3. Si ya toca: actuar y anotar
4. Seguir con lo demás

Un temporizador de hardware cuenta siempre; millis() solo lo mira.
Los timers son el tema de la semana 3.

¿Y cuando millis() da la vuelta?

248	250	252	254	0	2	4	6
	tAnterior			*reloj de 8 bits: 255 → 0*		ahora	

✓ Restar

ahora − tAnterior
= 4 − 250 = 10 (módulo 256)
Es justo el tiempo que pasó.

✗ Sumar y comparar

millis() > tAnterior + PERIODO
La suma da la vuelta y la condición se cumple antes de tiempo.

Ejemplo: dos LEDs a ritmos distintos

```
constexpr uint8_t  LED_A = 16, LED_B = 17;
constexpr uint32_t PERIODO_A = 500, PERIODO_B = 130;  // ms
uint32_t tA = 0, tB = 0;
bool estadoA = false, estadoB = false;

void setup() { pinMode(LED_A, OUTPUT); pinMode(LED_B, OUTPUT); }

void loop() {
  uint32_t ahora = millis();
  if (ahora - tA >= PERIODO_A) {
    tA += PERIODO_A; estadoA = !estadoA; digitalWrite(LED_A, estadoA);
  }
  if (ahora - tB >= PERIODO_B) {
    tB += PERIODO_B; estadoB = !estadoB; digitalWrite(LED_B, estadoB);
  }
}
```

Montaje

LEDs en GPIO16 y GPIO17
330 Ω a GND

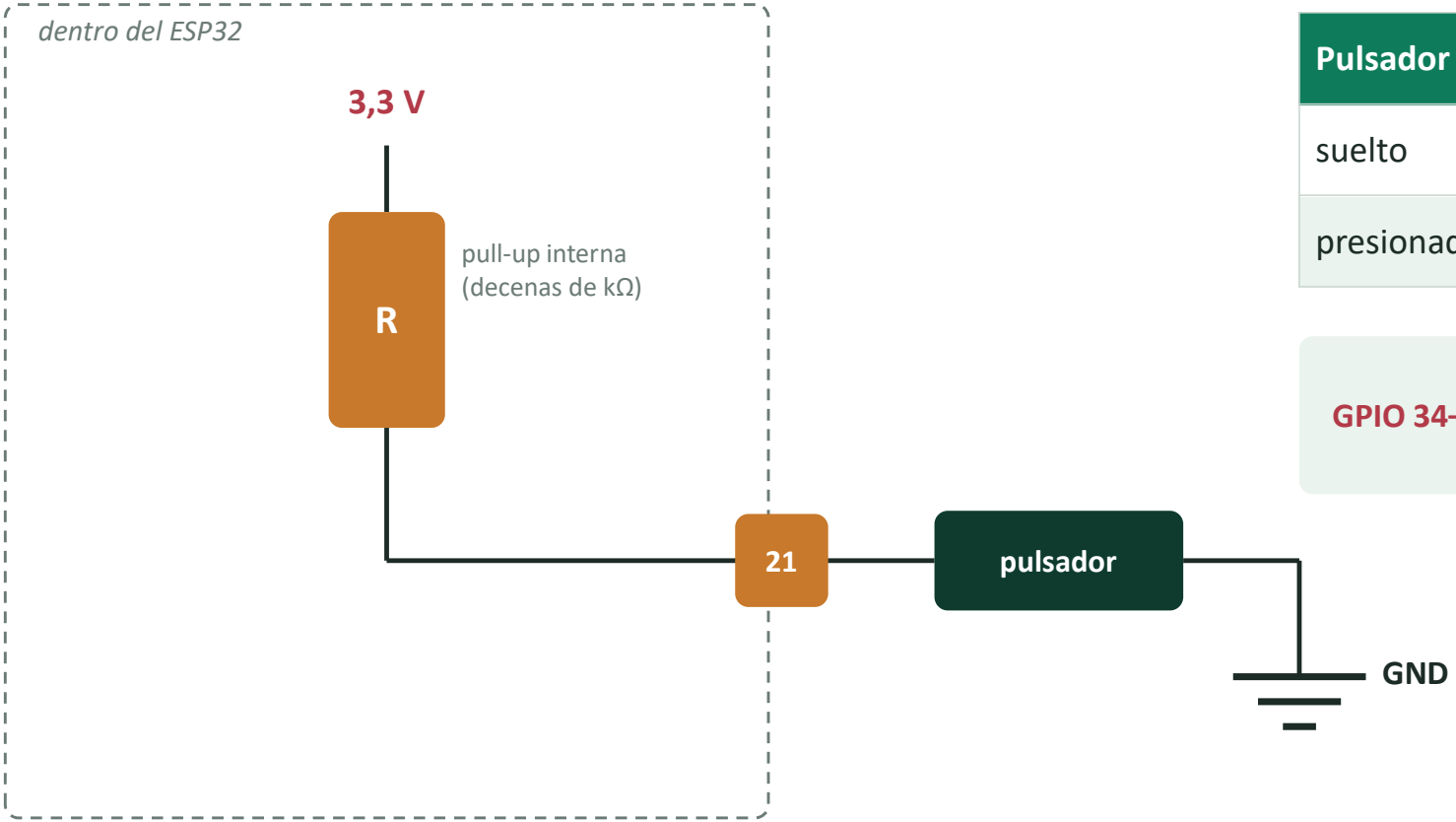
El estado vive en variables, no en el pin.

3

Parte 3 · Leer un pulsador bien

Pull-up, rebote y flanco

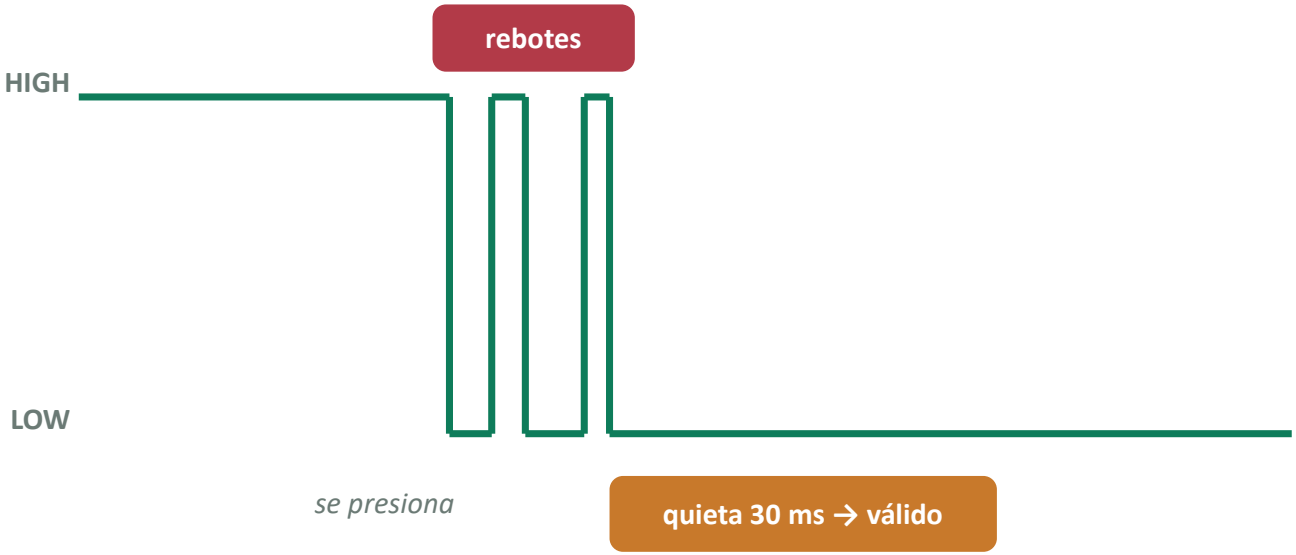
Pull-up: darle al pin un nivel definido



Pulsador	Lectura
suelto	HIGH (1)
presionado	LOW (0)

GPIO 34–39: sin pull-up interna

El rebote: la pelota que no se queda quieta



Antirrebote:
aceptar el cambio solo si la señal se queda quieta entre 20 y 50 ms.

Solución tentadora pero mala: delay(200) después de leer. Vuelve a bloquear el programa.

Nivel frente a flanco

flanco de bajada: UNA vez



nivel LOW: miles de vueltas de loop()

Un toque humano dura ~200 ms

✗ Cuenta por nivel

```
if (digitalRead(BTN) == LOW) contador++;
```

✓ Cuenta por flanco

Actuar solo cuando el estado pasa de HIGH a LOW.

Código: antirrebote y flanco

```
constexpr uint8_t  PIN_BTN = 21;
constexpr uint32_t T_REBOTE = 30;                // ms
bool lecturaAnterior = HIGH, estadoEstable = HIGH;
uint32_t tUltimoCambio = 0;  uint16_t pulsaciones = 0;

void setup() { Serial.begin(115200); pinMode(PIN_BTN, INPUT_PULLUP); }

void loop() {
  bool lectura = digitalRead(PIN_BTN);
  if (lectura != lecturaAnterior) {              // se movió: reiniciar
    tUltimoCambio = millis(); lecturaAnterior = lectura;
  }
  if (millis() - tUltimoCambio >= T_REBOTE && lectura != estadoEstable) {
    estadoEstable = lectura;                     // cambio real
    if (estadoEstable == LOW) {                  // flanco de bajada
      pulsaciones++; Serial.println(pulsaciones);
    }
  }
}
```

1. leer
2. ¿se movió?
3. ¿quieta 30 ms?
4. ¿flanco?

Todo junto: la estructura de un juego de luces

```
void loop() {
  // 1) Entradas
  if (botonPresionado()) velocidad = (velocidad + 1) % N_VEL;

  // 2) Lógica temporizada
  uint32_t ahora = millis();
  if (ahora - tPaso >= PERIODOS[velocidad]) {
    tPaso = ahora;
    posicion = (posicion + 1) % N_LEDS;
  }

  // 3) Salidas
  for (uint8_t i = 0; i < N_LEDS; i++)
    digitalWrite(LEDS[i], i == posicion);
}
```

Entradas

botón con antirrebote y flanco

Lógica

estado + millis()

Salidas

LEDs según el estado

Resumen

1 **Tipos**

int cambia de tamaño: usar uint8_t, uint32_t…

2 **Literales y máscaras**

1UL << n · cast tras ~ · desbordamiento sin signo da la vuelta

3 **Tiempo**

millis() + resta: ahora − tAnterior >= PERIODO

4 **Pulsador**

INPUT_PULLUP · antirrebote 30 ms · actuar en el flanco

Control de aprendizaje 1

20 min

en el EVA · individual

Cubre

- Registros GPIO del ESP32 y pines seguros
- setup(), loop() y el superlazo
- Tipos de datos, desbordamiento y máscaras
- millis() y el pulsador

Tarea: preparar la AP2

1	**Leer**	La guía de la AP2 en el EVA.
2	**Traer**	La placa ESP32 del grupo y un cable USB de datos.
3	**Instalar**	Arduino IDE 2.x + core esp32 3.x (si no lo hicieron).
4	**AA 1-2**	Revisar el catálogo de temas del proyecto de unidad en el EVA.

Viernes: AP2 · Juego de luces interactivo con ESP32. Los grupos sin placa trabajan en Wokwi.